Updated Code Review — Fortune Credit Data Analysis

Review date: 7 October 2026

Project: Fortune Credit: Seasonal Credit Risk Analysis

Scope: Local source code, bundled simulated dataset, prior review-response document, dashboard behavior, and static reporting.

Outcome: The bundled sample works, but input-validation and cache behavior can produce misleading results when the input changes.

Overall assessment

This is a small Streamlit, Pandas, and SQLite analysis case study, with Matplotlib charts and Excel exports through OpenPyXL. It should be assessed as a simulated-data demonstration, not as an operational lending or credit-decision system.

The earlier changes made useful improvements:

- Data cleaning is centralized in data_processing.py.
- Harvest gaps remain numeric before SQL aggregation.
- Dashboard metrics distinguish Repaid, Late, and Defaulted.
- Charts separate percentages from sample counts.
- The README identifies the data as simulated and avoids claiming statistical significance or causality.

The remaining weakness is that several validation checks only warn or silently coerce values. Invalid rows can still affect the reported statistics. The README's description of validation is therefore stronger than the implemented behavior.

Findings

1. Medium — Negative harvest gaps enter the wrong timing group

References: app.py, line 85, clean_data.py, line 30, data_processing.py, line 48.

The cleaner adds a Negative_Harvest_Gap flag, but neither harvest aggregation excludes those rows. The first SQL branch accepts every value <= 30, including negative values.

Reproduced: A loan disbursed on 15 January 2026 with an expected harvest on 1 January 2026 has a gap of −14 days. It was classified as 0-30 Days, contributing to that group's repayment rate.

Impact: Loans issued after harvest are presented as loans issued zero to thirty days before harvest. This changes the meaning of the comparison.

Recommended correction: Adopt an explicit policy for negative gaps: quarantine/exclude them with visible exclusion counts, or report a separate after-harvest group. Apply the same policy to the dashboard and static reports. Test boundaries −1, 0, 30, 31, 90 and 91 days.

2. Medium — Duplicate and missing loan IDs remain in the analysis

Reference: data_processing.py, line 18.

Duplicate IDs generate a Python warning but are retained. Missing loan IDs have no explicit validation check.

Reproduced: A synthetic dataset with one repaid loan and one defaulted loan should produce a 50% repayment rate when each loan is counted once. Repeating the repaid row yielded three counted rows, a 66.7% repayment rate, and a 33.3% default rate. A separate row with a missing ID was accepted without a warning and counted as a loan.

Impact: The reported loan count can be a row count rather than a count of valid, distinct loans.

Recommended correction: Require nonblank identifiers and define how exact duplicates and conflicting records are handled. Do not silently keep an arbitrary record when the same ID has different outcomes. Reject or quarantine invalid rows and expose their counts in the output.

3. Medium — Unknown repayment statuses distort denominators

References: data_processing.py, line 24, app.py, line 91.

Unexpected repayment statuses trigger a warning but remain in the data. SQL includes them in COUNT(*), while the repayment/default/late numerators only recognize known statuses.

Reproduced: One repaid loan and one loan with status Unknown produced a 50% repayment rate. The total loan count was two, but the three named status counts accounted for only one loan.

Impact: Missing or unrecognized outcomes reduce reported rates without making the unknown population visible. Users cannot reconcile the displayed totals.

Recommended correction: Normalize supported spellings and whitespace before validation. Then either reject/quarantine unsupported values with visible counts or introduce an explicit unknown-outcome category and document the denominator for each rate. The same policy must govern both reporting paths.

4. Medium — Invalid farmer dates silently disappear from timing analysis

References: data_processing.py, lines 39–40, app.py, line 93.

pd.to_datetime(..., errors='coerce') turns malformed dates into NaT. The harvest query then excludes their null gaps without distinguishing invalid farmer dates from legitimate non-agricultural missing harvest dates.

Reproduced: A farmer row with Expected_Harvest_Date = 'not-a-date' remained in portfolio totals, raised no warning, and produced no harvest-timing result.

Impact: The timing analysis can describe a smaller, potentially different population without disclosing why loans were excluded.

Recommended correction: Preserve validation flags for malformed or missing required dates. Validate requirements by customer type. Report the number of farmer loans eligible for timing analysis and the number excluded for each reason.

5. Medium — Nonnumeric loan amounts crash processing

Reference: data_processing.py, line 21.

The cleaner compares Loan_Amount to zero before establishing that the column is numeric.

Reproduced: A synthetic amount of KES 1,000 raised TypeError: Invalid comparison between dtype=str and int.

Impact: An invalid or formatted amount can prevent both the dashboard and static report from loading. Negative amounts also remain available to influence the median used for imputation.

Recommended correction: Define accepted amount formats and parse them explicitly. Distinguish missing values from invalid values, and validate finite/nonnegative values before computing imputation statistics. Handle an entirely missing/invalid amount column explicitly rather than claiming successful imputation.

6. Medium — Dashboard caching hides CSV updates

Reference: app.py, line 21.

get_clean_data() is cached without arguments representing the source file's contents or version, and without a refresh mechanism. Changes to the CSV alone do not invalidate that cached result on an ordinary rerun.

Reproduced: The dashboard initially showed ten loans. After an eleventh synthetic row was appended to the temporary CSV and the application reran, it still showed ten loans.

Impact: Users can view stale totals after updating the dataset. Static reports can also use newer data than an already-running dashboard.

Recommended correction: Include a content fingerprint or suitable file-version value in the cache key, or offer an explicit refresh action that clears the relevant cache. Display source/version information so outputs can be reconciled.

Additional improvements

These are maintenance and communication recommendations, not additional reproduced high-severity defects:

- Pin or constrain tested dependency versions and document the supported Python version. The current requirements list is entirely unversioned.
- Add automated regression tests for the cases above and keep synthetic fixtures separate from the demonstration dataset.
- Put the simulated-data and small-sample limitation directly in the dashboard and exported reports, not only in the repository README.
- Share aggregation logic between dashboard and static-report code so validation policies and metric definitions cannot drift.

• Update the prior implementation report to distinguish warning-only checks from enforced validation and to describe what is actually visible in each output.

Validation performed

Checks used an isolated temporary dependency environment and synthetic inputs. Dashboard and report execution used a temporary project copy so the original dataset and generated outputs were not modified.

Check	Result
Bundled dataset harvest analysis	README figures confirmed: 31–90 days = 2/2 repaid (100%); over 90 days = 2/5 repaid (40%)
Negative harvest gap	Reproduced incorrect inclusion of −14 days in the 0–30 group
Duplicate loan ID	Reproduced changed rates from repeated rows
Missing loan ID	Reproduced acceptance without an identity warning
Unknown repayment status	Reproduced denominator/status-count inconsistency
Invalid farmer harvest date	Reproduced silent exclusion from timing analysis
Nonnumeric amount	Reproduced TypeError
CSV change followed by dashboard rerun	Reproduced stale cached total of 10 after source grew to 11 rows
Dashboard customer filter	Boda Rider filter displayed two loans: one repaid, one late, zero defaulted
Static reporting	Excel workbook and both chart files generated successfully in the temporary copy

This was a local code and behavior review, not a review of the deployed dashboard, hosting configuration, statistical model validity, or real customer data. The passing sample flow does not establish production readiness.

Recommended order of work

1. Define validation and quarantine/rejection rules for identities, statuses, amounts and dates.
2. Correct harvest eligibility and timing categories, and display sample/exclusion counts.
3. Fix cache invalidation and identify the dataset version used in each output.
4. Consolidate aggregation logic and add regression tests.
5. Align documentation and exported-report limitations with the implemented behavior.

No application code or source data was changed during the review.